Author: Imama Zubair AI & DS Intern @ NetixSol

AFL Assistant: Executive Report

Product Goal

The goal of this project was to build an AFL-only assistant that can answer AFL questions, look up real player and team statistics, and make match and player predictions. It is designed to stay within the AFL domain and refuse unrelated questions. The assistant is hopefully built as a template for a client product, similar to a Web3Geeks deployment. It clearly separates factual information from machine learning predictions and explains that predictions are estimates, not guarantees.

Architecture

The assistant is built using LangGraph. Every user question first goes through a rule-based router that uses keywords and entity matching instead of an LLM to decide what to do.The router sends the request to one of four paths:

1. **Retrieval:** Looks up player statistics and team head-to-head records from the feature tables.
2. **Prediction:** Uses the Day 2 machine learning models through predict.py to predict match winners or top players.
3. **Factual:** Answers general AFL questions using the Day 3 chat agent, which has an AFL-only system prompt.
4. **Off-topic:** Politely refuses questions that are not related to AFL.

Before showing any retrieval or prediction result, the assistant checks whether enough information is available. If a team or player cannot be identified, it asks the user for clarification instead of guessing. Every prediction includes a probability, a disclaimer explaining that it is only a prediction, and a short explanation of the features that influenced the result.

For Day 5, several **improvements** were added:

- Added consistent error handling across every graph node.
- Added a 12-second timeout for slow tool and LLM calls.
- Added prompt injection detection and logging.
- Added stronger responses after repeated off-topic questions.
- Added structured logging for every request, including query, detected intent, tools, latency, errors.
- Wrapped the assistant behind a FastAPI /chat endpoint and created an optional Streamlit interface for demonstrations.

Evaluation Results

The assistant was tested using 30 cases across five categories. Some factual question tests require a live LLM connection. If API credentials were not available, those cases were reported honestly instead of marking them as passed.

Category	Cases	Passed	Pass Rate	Notes
Scope guardrails	9	9	9/9	Three prompt injection attempts tested successfully
Retrieval	4	4	4/4	Team head-to-head and player statistics
Prediction sanity	9	9	9/9	Probabilities and disclaimers verified
Multi-turn coherence	4	4	4/4	Fixed after identifying a state management bug
Factual Q&A	4	4	4/4	esponses timed out when the LLM quota was exhausted

The match winner model was also compared against a simple ladder-position baseline using the most recent 860 matches (2020–2025).

- **Trained model:** 66.3% Accuracy, 0.734 F1
- **Ladder-position baseline:** 65.8% Accuracy, 0.694 F1

The trained model performs slightly better than the simple baseline. While the accuracy improvement is small, the higher F1 score shows that its predictions are more balanced and consistent. This makes it suitable for a demonstration system while still making it clear that predictions are not guaranteed.

Weakest Category: The weakest area during development was multi-turn conversations. Initially, only 2 out of 4 test cases passed.
The **issue** was caused by run_turn() resetting the conversation state on every request, which removed previously stored entities before the router could use them. This broke follow-up questions such as "Who's more likely to win between them?" because the assistant no longer knew which teams the user was referring to.
Fix: The issue was fixed by only creating a full state on the first message in a conversation. Later messages now update only the necessary fields, allowing previous context to remain available. After this fix, all four multi-turn tests passed.

Limitations

- **Data recency:** Predictions use the latest available feature tables. There is no live fixture feed, so the assistant only knows the most recent data that has been loaded.
- **Model accuracy:** AFL matches are difficult to predict. A model with around 66% accuracy is useful for identifying favourites, but it will still be wrong about one out of every three matches.
- **Guardrails:** The router uses keywords instead of semantic understanding. In rare situations, an unrelated question written using AFL terms could still reach the factual chat agent. The AFL-only system prompt provides an additional layer of protection.

- **Rate limiting:** Abuse tracking is stored only in memory and is suitable for demonstrations, but not for a production deployment.

Recommended Next Steps

1. Implement the weekly retraining process described in the monitoring checklist so the models stay up to date throughout the season.
2. Add proper per-user rate limiting before deploying the assistant publicly.
3. Re-run the factual Q&A evaluation once LLM quota resets
4. Add a small vector-based FAQ system for common AFL questions to reduce LLM cost and improve response time